

Computer Graphics Project Report

Raytracer Implementation

Myriam Dardour

May 18, 2026

1 Introduction

This project implements a raytracer progressively extended over four labs. Starting from the provided C++ skeleton, I implemented the missing rendering components: (Lab 1) sphere intersection, direct lighting, shadows, mirror reflection, (Lab 2) Monte Carlo indirect lighting, antialiasing, (Lab 3) triangle mesh intersection, and (Lab 4) BVH acceleration.

The goal of the report is not to restate the derivations from the lecture notes, but to show how the implementation evolved and how each feature was validated visually and through render-time comparison.

2 Direct Lighting, Shadows and Mirror Reflection (Lab 1)

Lab 1 established the basic raytracing pipeline. For each pixel, I generate a primary ray from the camera through the image plane, intersect it with the scene, and shade the closest visible object. As we can see in Figure 1 the scene is made of a central sphere and large surrounding spheres used to approximate the floor, ceiling and walls. This simple environment was useful for debugging because the colored walls make reflection errors easy to see.

I implemented ray-sphere intersection and ray-scene intersection by keeping the closest positive hit. This "closest-hit" logic is essential: without it, objects behind the visible surface can incorrectly affect the pixel color.

Diffuse surfaces are shaded with direct Lambertian lighting from a point light source. I also added shadow rays: after a surface point is found, a ray is sent toward the light, and the direct contribution is removed if another object is hit before the light. I had to offset secondary rays by $P + \varepsilon N$ to avoid self-intersection artifacts, which otherwise produced incorrect dark pixels on the surface as shown in the lecture.

Mirror reflection was implemented recursively. If the hit object is marked as a mirror, the renderer launches a new ray in the reflected direction, and uses the returned color as the mirror color. I limited the recursion depth (max light bounces = 8) to avoid infinite reflection paths.

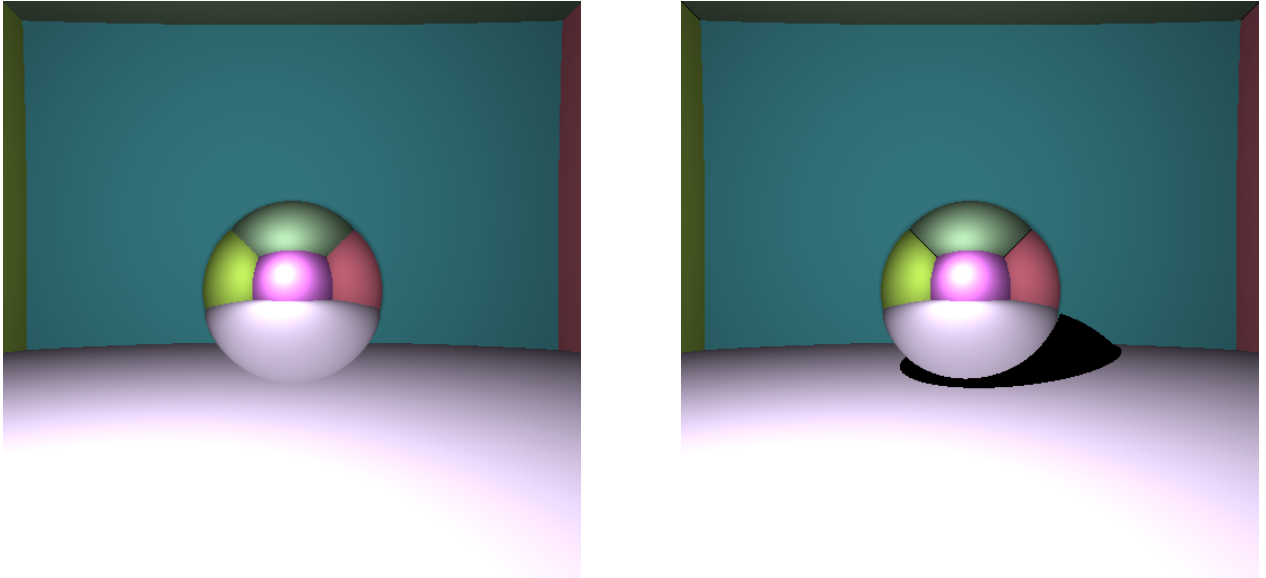


Figure 1: Lab 1 results. Left: Mirror sphere reflecting the colored environment. Right: Added shadow. Resolution: 512×512 . Gamma = 2.2

These images validate the main Lab 1 features: closest intersections, direct lighting, shadow rays, and mirror reflection.

3 Indirect Lighting and Antialiasing (Lab 2)

Lab 2 extends the renderer from a direct-light raytracer to a path tracer. The main change is that diffuse surfaces now receive not only direct light from the point source, but also indirect light arriving after previous bounces in the scene.

At each diffuse intersection, I sample a random direction above the surface using a cosine-weighted distribution. This is more efficient than uniform sampling because directions close to the normal contribute more to the Lambertian term. I then recursively call `getColor` in that direction and multiply the returned radiance by the surface albedo. To orient the sampled hemisphere correctly at each surface point, I construct a local tangent frame (T_1, T_2, N) and convert the sampled direction to world coordinates. As in Lab 1, I offset the new ray origin by $P + \epsilon N$ to avoid numerical bugs.

Antialiasing was implemented at the camera sampling stage. In Lab 1, each pixel was represented by one ray through its center, which can create staircase artifacts when geometry boundaries fall between pixels. In Lab 2, I sample multiple camera rays around the pixel center using Gaussian offsets and average their contributions. This results in smoother contours as we can see in Figure 2 (left).

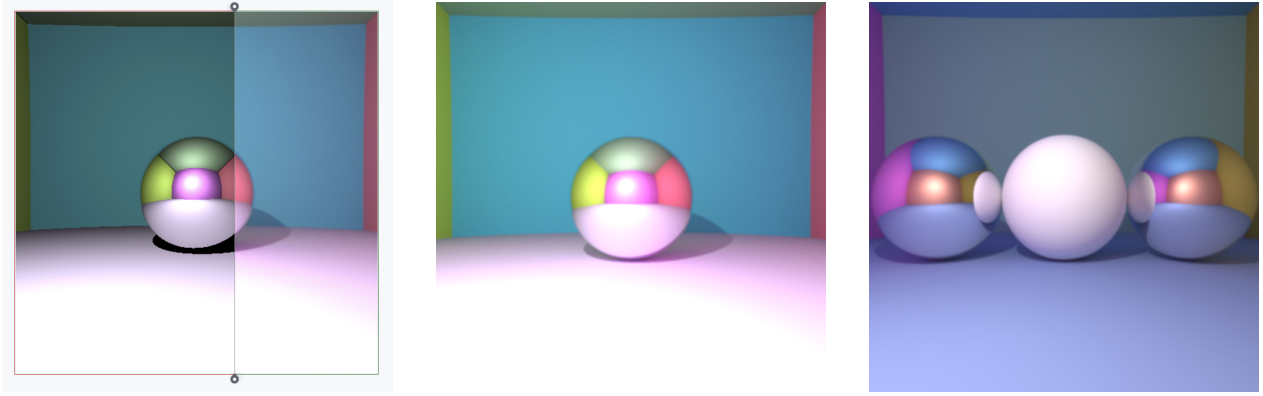


Figure 2: Lab 2 results showing Monte Carlo indirect lighting and antialiasing. **Left:** Comparison Lab 1 (left) vs Lab 2 (right) results. **Middle:** $I = 1E7$. **Right:** $I = 3E7$. For all Resolution: 512×512 . Number of paths per pixel: 1000. Maximum bounce depth: 8.

Compared to Lab 1, the most visible improvements are:

- Shadowed regions are no longer completely black: they receive secondary illumination from nearby surfaces and so they are much lighter.
- The contours are not harsh boxes looking like stairs, they appear much smoother. The image is now much more realistic.

4 Lab 3: Triangle Mesh Rendering

Lab 3 adds support for OBJ triangle meshes. Instead of being limited to analytic primitives such as spheres, the renderer can now display arbitrary geometry represented as a list of triangles like for example a cat.

For each triangle, I implemented ray-triangle intersection using the **Möller-Trumbore** algorithm from the lecture notes. The key implementation point was to reject hits outside the triangle by checking barycentric coordinates, and to keep only the closest valid triangle hit. Once a triangle is hit, I compute its geometric normal and use the same shading pipeline as before: direct light, shadows, indirect bounces and antialiasing.

I also added a first acceleration step using the global bounding box of the mesh. Before testing all triangles, the renderer checks whether the ray intersects the mesh bounding box. If it does not, the whole triangle loop is skipped. This does not change the rendered image, but it avoids many unnecessary triangle intersection tests.

To test our code we used the cat OBJ file. I applied the a scale and translation to make the model fit inside the scene. This step was necessary because OBJ model did not use the same size and origin as the renderer scene.

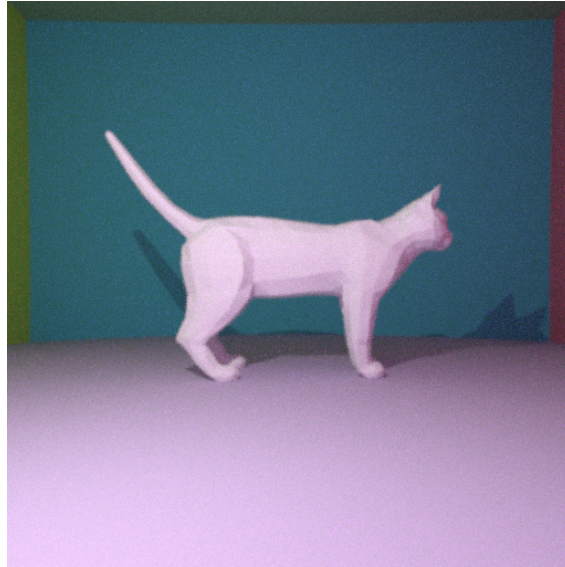


Figure 3: Lab 3 results. Cat model scaled by a factor 0.6 and translated by the vector (0, 10, 0) Resolution: 512×512 . Number of paths per pixel: 32. Maximum bounce depth: 8. Render time: **2:40.84** (with bounding box).

This result validates that the renderer is no longer restricted to simple mathematical shapes. Any OBJ mesh can now be inserted into the scene and shaded using the same material and lighting logic.

5 Lab 4: BVH Acceleration

Lab 4 focuses on performance. The Lab 3 mesh renderer works, but brute-force intersection is inefficient because every ray is tested against every triangle. This becomes expensive very quickly, especially with many paths per pixel. To solve this, we implement a Bounding Volume Hierarchy inside the `TriangleMesh` class.

Each BVH node stores an axis-aligned bounding box and a continuous range of triangle indices. During construction, I compute the bounding box of the node, select the longest axis, and split triangles according to their barycenter along that axis. This recursive partitioning creates a hierarchy where large parts of the mesh can be rejected with only one bounding-box test.

The recursion stops when the node contains at most 5 triangles or if the split would put all triangles on the same side. The second condition is important because in that case, this would not reduce the problem and could lead to infinite recursion.

During traversal, the ray first tests the node bounding box. If it misses, the whole subtree is discarded. If the node is a leaf, only its small triangle range is tested. I also keep track of the closest hit found so far, so boxes farther than the current best intersection can be ignored. This keeps the final image unchanged while greatly reducing the number of triangle tests.

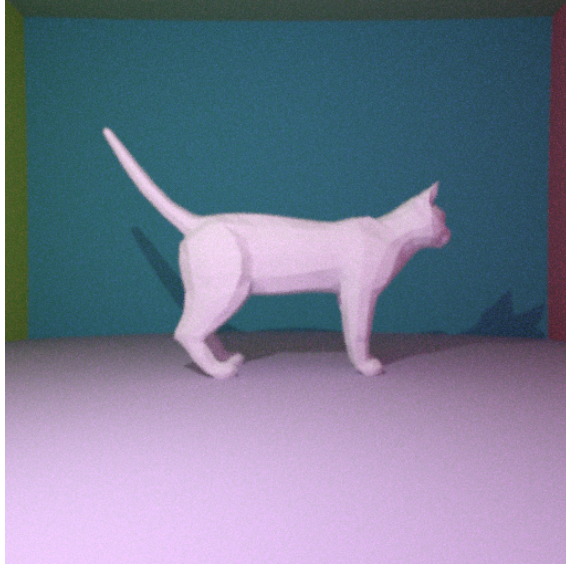


Figure 4: Lab 4 result. Resolution: 512×512 . Number of paths per pixel: 32. Maximum bounce depth: 8. Render time with BVH: **3.710**.

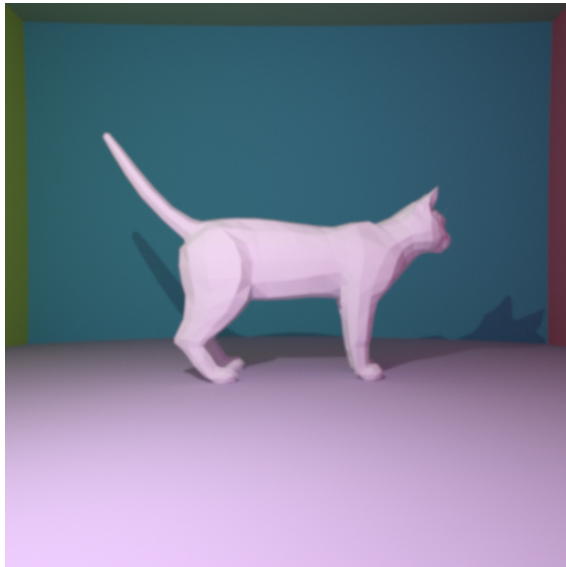


Figure 5: Lab 4 result. Resolution: 512×512 . Number of paths per pixel: 1000. Maximum bounce depth: 8. Render time with BVH: **1:58.69**.

6 Performance Comparison

The main purpose of the BVH is acceleration, not visual change. Therefore, I compared the brute-force Lab 3 mesh intersection with the BVH-accelerated Lab 4 version using the same scene, resolution, and number of paths per pixel.

Method	Intersection strategy	Image quality	Render time
Brute-force mesh	Test all triangles	Same result	2:40.84
BVH mesh	Test only visited leaves	Same result	3.710

Table 1: Performance comparison between brute-force mesh intersection and BVH traversal. The BVH accelerates rendering by skipping large triangle groups while preserving the same visual output.

The brute-force version took 2 min 40.84 s, i.e. 160.84 s, while the BVH version took only 3.710 s. This corresponds to a speedup of approximately

$$\frac{160.84}{3.710} \approx 43.4.$$

In other words, the BVH made the mesh rendering about $43\times$ faster while producing the same image. This confirms that the acceleration structure improves performance by reducing unnecessary triangle intersection tests, without changing the visual result.

7 Conclusion

Over the four labs, the renderer evolved from a simple sphere raytracer into a basic path tracer supporting mirror reflection, indirect diffuse illumination, antialiasing, OBJ triangle meshes, and BVH acceleration.

The most important implementation steps were not only the formulas themselves, but the numerical and structural details: keeping the closest positive intersection, offsetting secondary rays to avoid self-intersection, averaging enough Monte Carlo paths to reduce noise, integrating meshes through the same `Object` interface as spheres, and using BVH traversal to make mesh rendering practical.